COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 07/10/26 | **DATE SUBMITTED:** | 07/10/26 |
| **NAME:** | CRUZ, ELIJAH J. | **INSTRUCTOR:** | SIR JIMLORD QUEJADO |

**ACTIVITY NO. 1** **Hands-on Activity 1.1 Basic C++ Programming**

## 1. PROCEDURE OUTPUT

**ILO A: Create Code That Follows the Basic C++ Code Structure**

- Create a function that will take two numbers and display the sum.

```
#include <iostream>

int sumofTwoNum(int fNum, int sNum);

int main () {

    std::cout << "the sum of the two numbers is: " << sumofTwoNum(6,7)
<< std::endl;

    return 0;
}

int sumofTwoNum(int fNum, int sNum) {
    return fNum + sNum;
}
```

- Create a function that will return whether variable A is greater than variable B.

```
#include <iostream>

bool isGreaterThan(int variableA, int variableB);

int main() {
    int a = 11;
    int b = 10;
```

```
    std::cout << std::boolalpha;

    std::cout << "Is A greater than B? \n" << isGreaterThan(a, b) <<
std::endl;

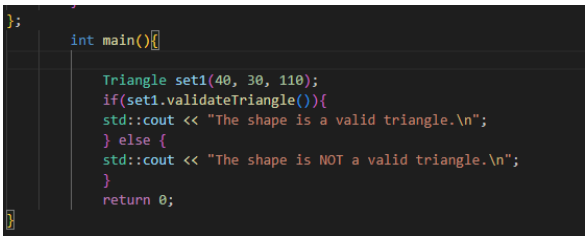
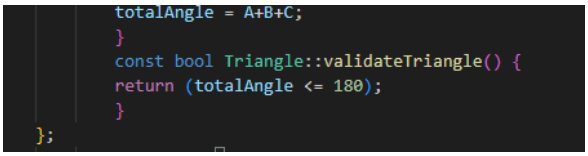
    return 0;
}

bool isGreaterThan(int variableA, int variableB) {
    return variableA > variableB;
}
```

- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success

**ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements**

| section | answer |
|---|---|
| 1. Header File Declaration Section | `#include <iostream>`<br>`class Triangle{`<br>`private:`<br>`double totalAngle, angleA, angleB, angleC;` |
| 2. Global Declaration Section | `class Triangle{`<br>`private:`<br>`double totalAngle, angleA, angleB, angleC;`<br>`public:`<br>`Triangle(double A, double B, double C);`<br>`void setAngles(double A, double B, double C);`<br>`const bool validateTriangle();` |
| 3. Class Declaration and Method Definition Section | `Triangle::Triangle(double A, double B, double C) {`<br>`angleA = A;`<br>`angleB = B;`<br>`angleC = C;`<br>`totalAngle = A+B+C;`<br>`}`<br>`void Triangle::setAngles(double A, double B, double C) {`<br>`angleA = A;`<br>`angleB = B;`<br>`angleC = C;`<br>`totalAngle = A+B+C;`<br>`}`<br>`const bool Triangle::validateTriangle() {`<br>`return (totalAngle <= 180);`<br>`}` |

| | |
|---|---|
| 4. Main Function |  |
| 5. Method Definition |  |
| 6. Table 1-1. C++ Structure Code for Answer | #include <iostream><br><br>class Triangle {<br>private:<br>double totalAngle, angleA, angleB, angleC;<br><br>public:<br>Triangle(double A, double B, double C);<br>void setAngles(double A, double B, double C);<br>bool validateTriangle() const; // Moved 'const' to the correct position<br>};<br><br>Triangle::Triangle(double A, double B, double C) {<br>angleA = A;<br>angleB = B;<br>angleC = C;<br>totalAngle = A + B + C;<br>}<br><br>void Triangle::setAngles(double A, double B, double C) {<br>angleA = A;<br>angleB = B;<br>angleC = C;<br>totalAngle = A + B + C;<br>}<br><br>bool Triangle::validateTriangle() const {<br>return (totalAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0);<br>}<br><br>int main() { |

| | |
|---|---|
| | // Driver code<br>Triangle set1(40, 30, 110);<br><br>if (set1.validateTriangle()) {<br>std::cout << "The shape is a valid triangle.\n";<br>} else {<br>std::cout << "The shape is NOT a valid triangle.\n";<br>}<br><br>return 0;<br>} |
| 7. Table 1-2. ILO B output observations and comments | #include <iostream><br><br>class Triangle {<br>private:<br>double totalAngle, angleA, angleB, angleC;<br><br>public:<br>Triangle(double A, double B, double C);<br>void setAngles(double A, double B, double C);<br>bool validateTriangle() const;<br>};<br><br>Triangle::Triangle(double A, double B, double C) {<br>angleA = A;<br>angleB = B;<br>angleC = C;<br>totalAngle = A + B + C;<br>}<br><br>void Triangle::setAngles(double A, double B, double C) {<br>angleA = A;<br>angleB = B;<br>angleC = C;<br>totalAngle = A + B + C;<br>}<br><br>bool Triangle::validateTriangle() const {<br>return (totalAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0);<br>}<br><br>// Creates the object, runs the validation, and prints |

| | |
|---|---|
| | the result<br>int main() {<br>Triangle set1(40, 30, 110);<br><br>if (set1.validateTriangle()) {<br>std::cout << "The shape is a valid triangle.\n";<br>} else {<br>std::cout << "The shape is NOT a valid triangle.\n";<br>}<br><br>return 0;<br>} |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Create a C++ program to swap the two numbers in different variables.

```
#include <iostream>

void swapNumbers(int &num1, int &num2);

int main() {
    int a, b;

    std::cout << "Enter first number (A): ";
    std::cin >> a;
    std::cout << "Enter second number (B): ";
    std::cin >> b;

    std::cout << "\nBefore swap: A = " << a << ", B = " << b <<
std::endl;

    swapNumbers(a, b);

    std::cout << "After swap:  A = " << a << ", B = " << b <<
std::endl;
```

```
    return 0;
}

void swapNumbers(int &num1, int &num2) {

    int temp = num1;
    num1 = num2;
    num2 = temp;
}
```

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```
#include <iostream>

double kTempF(double kelvin);

int main() {
   double val1;
   std::cout << "Write the temperature in Kelvin: \n";
   std::cin >> val1;

   std::cout << "The temperature in Fahrenheit is: " << kTempF(val1) << std::endl;

    return 0;
} double kTempF(double val1) {
   return (val1 - 273.15) * 1.8 + 32;

}
```

3. Create a C++ program that has a function that will calculate the distance between two points.

```
#include <iostream>

double DistanceSq(double x1, double y1, double x2, double y2);

int main() {
   double x1, y1, x2, y2;

   std::cout << "Enter coordinates for Point 1 (x1 y1): ";
   std::cin >> x1 >> y1;

   std::cout << "Enter coordinates for Point 2 (x2 y2): ";
```

```
    std::cin >> x2 >> y2;

    std::cout << "Squared Distance: " << DistanceSq(x1, y1, x2, y2) << std::endl;

    return 0;
}

double DistanceSq(double x1, double y1, double x2, double y2) {
    double dx = x2 - x1;
    double dy = y2 - y1;
    return (dx * dx) + (dy * dy);
}
```

4. Modify the code given in ILO B and add the following functions:
   a. A function to compute for the area of a triangle

```
#include <iostream>

// 1. Class Declaration
class Triangle {
private:
    double totalAngle, angleA, angleB, angleC;

public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    bool validateTriangle() const;

    double computeArea(double base, double height) const;
};

Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}
```

```
bool Triangle::validateTriangle() const {
    return (totalAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0);
}

// a. Simple Area calculation (Base * Height / 2)
double Triangle::computeArea(double base, double height) const {
    return 0.5 * base * height;
}

int main() {
    Triangle set1(40, 30, 110);

    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";

        // Print the result of the area function directly
        std::cout << "Area: " << set1.computeArea(10, 5) << "\n";
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }

    return 0;
}
```

b. A function to compute for the perimeter of a triangle

```
#include <iostream>

// 1. Class Declaration
class Triangle {
private:
    double totalAngle, angleA, angleB, angleC;

public:
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    bool validateTriangle() const;

    double computePerimeter(double sideA, double sideB, double sideC) const;
};

Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
```

```
}

void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

bool Triangle::validateTriangle() const {
    return (totalAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0);
}

// b. Perimeter calculation
double Triangle::computePerimeter(double sideA, double sideB, double sideC) const {
    return sideA + sideB + sideC;
}

int main() {
    Triangle set1(40, 30, 110);

    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";

        std::cout << "Perimeter: " << set1.computePerimeter(7, 6, 10) << "\n";
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }

    return 0;
}
```

Provide the following:

- Summary of lessons learned
  - Basic concepts and function prototyping. In addition, how to use classes and objects.
- Analysis of the procedure
  - The procedures in the activity involved learning how to use function prototypes, class and objects.
- Analysis of the supplementary activity
  - The supplementary activity involves using function prototypes, class and objects and applying them to create programs aligned with the necessary requirements.
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?
  - The activity proved to be quite difficult, especially number 4 in the supplementary activity to learn as it relies on setting the correct parameters and variables in order for the process to return and proceed with the prototype function. For areas of improvement, I need to work more on familiarizing myself on code structures and on different data types as I often struggle with what structure and ideas to use to fulfill the requirements needed.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:**

*Gemini*

**Purpose of Use:**

*Explanation of functions and code assistance in setting up the parameters.*

**Extent of Use:**

*Gemini was used to explain how function prototyping works and assistance in fixing errors within my code. I confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.